Python Tutorial For Beginners in Hindi | Complete Python Course

Channel: CodeWithHarry
Video: https://www.youtube.com/watch?v=UrsmFxEIp5k
Language: English · **Words:** 102,913 · **Duration:** 10:53:35
Generated 17 Oct 2026 with LectureLens

Executive Summary

This tutorial provides a comprehensive, beginner-friendly introduction to Python programming designed to help learners acquire practical coding skills for software development and job readiness. It covers Python installation, setting up Visual Studio Code, and core programming fundamentals including modules, comments, variables, operators, input handling, and data types. Students learn essential data structures—strings, lists, tuples, dictionaries, and sets—along with conditional statements and an introduction to loops. Each theoretical section is paired with hands-on practice problems, demonstrating how to use modern developer tools and AI assistants like ChatGPT effectively to aid learning and problem-solving.

Key Points

• **Python's Simplicity**: Python was created by Guido van Rossum in 1991; its clear, English-like syntax makes it an ideal first programming language for beginners.
• **Development Environment Setup**: Setting up Python involves enabling the `python.exe to PATH` option during installation and using Visual Studio Code (VS Code) as an code editor with extensions and auto-save enabled.
• **Modules and Pip**: Pip is Python's package manager used to install external libraries (e.g., `flask`, `pyjokes`, `pyttsx3`), allowing developers to reuse pre-written code in their projects.
• **Variables and Basic Data Types**: Python supports integers (`int`), floating-point numbers (`float`), strings (`str`), booleans (`bool`), and `NoneType`, with strict naming rules for variable identifiers.
• **Data Mutability**: Strings and tuples are immutable (cannot be changed after creation), whereas lists and dictionaries are mutable (can be modified in place).
• **Dictionaries and Sets**: Dictionaries store key-value pairs for fast lookups, while sets store unique, unindexed, non-repetitive elements and support mathematical set operations (union and intersection).
• **Control Flow**: Conditional statements (`if`, `elif`, `else`) enable multi-way decision-making based on evaluation of conditions, relying strictly on whitespace indentation to define code blocks.
• **AI-Assisted Learning**: AI tools like ChatGPT can be leveraged responsibly during

learning to explain code output, review practice problem solutions, and highlight potential improvements.

Study Notes

1. Programming Fundamentals & Environment Setup

What is Programming?

Programming is a mechanism to communicate with computers using specific programming languages like Python, C, C++, Java, or Go. Computers require explicit instructions written in code, which an interpreter converts into low-level machine instructions to execute on hardware (see [02:41]).

Python Features & History

- Created by Guido van Rossum in 1991 and named after the comedy series *Monty Python's Flying Circus* (see [09:25]).
- High-level language, simple to learn, readable like plain English, free, open-source, and highly portable across Windows, Mac, and Linux systems (see [03:44], [05:20]).
- Widely used in Machine Learning, Data Analytics, Web Development, and Scripting due to extensive library support.

Software Installation & Configuration

- **Python Installation**: Download the Python setup (e.g., version 3.12.3) from the official website and check the option **"Add python.exe to PATH"** before proceeding with installation (see [07:24]).
- **Editor Setup**: Install Visual Studio Code (VS Code), enable all suggested check boxes (such as "Open with Code"), enable auto-save, and install the official Python extension (see [08:24], [09:25]).

––

2. Chapter 1: Modules, Comments, and Pip

Modules and Package Management

- **Module**: A file containing code written by another developer that can be imported into your program to perform tasks without writing logic from scratch (see [13:27]).
  - *Built-in Modules*: Included natively with Python (e.g., `os`).
  - *External Modules*: Installed via `pip` (Python's package manager).
- **Pip Commands**:

```bash
pip install flask
pip install pyjokes
pip install pyttsx3
```

```
```

- **Using Modules Example**:

```python
import pyjokes

joke = pyjokes.get_joke()
print(joke)
```
 (see [15:28])

Python REPL & Calculator

- Running `python` directly in the terminal opens the **REPL** (Read-Evaluate-Print Loop), allowing instant evaluation of expressions (e.g., `5 + 6` yields `11`) (see [17:34], [18:05]).

Comments

- **Single-line Comments**: Start with `#` symbol. Keyboard shortcut in VS Code: `Ctrl + /` (see [20:39]).
- **Multi-line Comments**: Enclosed in triple quotes (`'''` or `"""`) (see [22:41]).

–-

3. Chapter 2: Variables and Data Types

Variables & Core Data Types

A variable is a named container pointing to a memory location storing a value (see [35:55]).

Data Type	Description	Example
Integer (`int`)	Whole numbers without decimals	`a = 1`
Float (`float`)	Numbers containing decimal points	`b = 5.22`
String (`str`)	Sequence of characters in quotes	`c = "Harry"`
Boolean (`bool`)	Truth values (`True` or `False`)	`d = True`
None (`NoneType`)	Used to indicate absence of value	`e = None`

Naming Rules for Variables

1. May contain alphabets, digits, and underscores (`_`).
2. Must start only with an alphabet or an underscore.
3. Cannot start with a digit (e.g., `1Sameer` is invalid).
4. No white spaces or special characters (like `@`, `$`) are allowed (see [43:10], [44:07]).

Operators

- **Arithmetic Operators**: `+`, `-`, `*`, `/`, `%` (modulo operator returning remainder) (see [47:02], [1:07:39]).
- **Assignment Operators**: `=`, `+=`, `-=`, `*=`, `/=` (see [48:02]).

• **Comparison Operators**: `==`, `!=`, `>`, `<`, `>=`, `<=` (always return boolean `True` or `False`) (see [49:33]).
• **Logical Operators**: `and` (True if both operands are True), `or` (True if at least one operand is True), `not` (inverts truth value) (see [52:33], [56:03]).

Typecasting & Input Function

• `type()` returns the data type of a variable.
• Functions like `int()`, `float()`, `str()` convert valid data types into another (see [57:33], [59:33]).
• **`input()` Function**: Receives input from the user. It **always** returns user input as a string (`str`) and must be explicitely typecast for numeric operations (see [1:02:07], [1:05:39]):

```python
a = int(input("Enter number 1: "))
b = int(input("Enter number 2: "))
print("Sum is:", a + b)
```

--

4. Chapter 3: Strings

Creating Strings & Immutability

• Strings can be enclosed in single (`'`), double (`"`), or triple (`'''` or `"""`) quotes (see [1:16:41]).
• **Strings are Immutable**: String contents cannot be altered in place once created (see [1:17:45], [1:48:08]).

String Slicing

• Syntax: `string[start:end:skip]`
• Indexing starts at `0` for forward indexing, and `-1` for negative indexing (from the right) (see [1:18:15], [1:22:23]).
• The `start` index is included, while the `end` index is excluded.
• Leaving `start` blank defaults to `0`; leaving `end` blank defaults to the length of the string (see [1:24:26]).
• **F-Strings**: Formatted string literals allow variables directly inside string curly braces: `f"Good afternoon {name}"` (see [1:41:37]).

Useful String Methods

• `len(s)`: Returns total length of string.
• `s.endswith("str")` / `s.startswith("str")`: Returns boolean.
• `s.capitalize()`: Capitalizes first character.
• `s.find("word")`: Returns starting index of substring or `-1` if not found.
• `s.replace("old", "new")`: Replaces occurrences of a substring with a new substring.

Escape Sequence Characters

- `\n`: Inserts a newline.
- `\t`: Inserts a tab space.
- `\"` / `\'`: Escapes quotes inside string literals (see [1:36:35], [1:38:05]).

--

5. Chapter 4: Lists and Tuples

Lists (Mutable Collections)

- Lists store an ordered set of elements of any data type using square brackets `[]` (see [1:51:38]).
- **List Methods**:
  - `l.append(val)`: Adds `val` at the end of the list.
  - `l.sort()`: Sorts list in ascending order in place.
  - `l.reverse()`: Reverses the current order of the list.
  - `l.insert(index, val)`: Inserts `val` at specified `index`.
  - `l.pop(index)`: Removes and returns element at given `index`.
  - `l.remove(val)`: Removes the first occurrence of `val`.

Tuples (Immutable Collections)

- Tuples store an ordered set of items using parentheses `()` and cannot be changed after creation (see [2:06:54], [2:09:29]).
- **Single Element Tuple**: Must end with a comma, e.g., `a = (1,)` (see [2:08:28]).
- **Tuple Methods**:
  - `t.count(val)`: Returns number of times `val` appears in tuple.
  - `t.index(val)`: Returns index of the first occurrence of `val`.
  - `len(t)`: Returns total length of tuple.

--

6. Chapter 5: Dictionaries and Sets

Dictionaries

- Dictionaries store data in `key: value` pairs enclosed in `{}` (see [2:26:40]).
- **Properties**: Unordered, mutable, indexed by keys, and cannot contain duplicate keys (see [2:30:38]).
- **Dictionary Methods**:
  - `d.items()`: Returns list of key-value tuple pairs.
  - `d.keys()`: Returns all key names.
  - `d.values()`: Returns all values.
  - `d.update({"key": "val"})`: Updates dictionary with new/modified key-value pairs.
  - `d.get("key")`: Returns value of key or `None` if key does not exist (whereas

`d["key"]` throws a `KeyError` if key is missing) (see [2:35:40], [2:36:10]).

Sets

• Sets store unique, non-repetitive elements inside `{}` (see [2:38:40], [2:41:40]).
• **Empty Set Syntax**: Must use `e = set()`. Writing `e = {}` creates an empty dictionary (see [2:39:10], [2:40:40]).
• **Properties**: Unordered, unindexed, elements must be hashable/immutable (e.g., lists cannot be stored in sets) (see [2:44:10], [3:08:32]).
• **Set Methods & Operations**:
  • `s.add(val)`: Adds an element.
  • `s.remove(val)`: Removes specified element.
  • `s.union(s2)`: Returns combined set of elements from both sets.
  • `s.intersection(s2)`: Returns set containing only overlapping elements (see [2:47:15], [2:48:18]).

–-

7. Chapter 6: Conditional Expressions

If, Elif, Else Ladder

• Conditionals execute specific blocks of code depending on evaluated boolean expressions (see [3:10:02], [3:11:32]).
• **Syntax & Indentation**: Python relies strictly on standard indentation (spaces/tabs) following a colon `:` to mark code blocks (see [3:12:32], [3:13:02]).

```python
if condition1:
# Code block 1
elif condition2:
# Code block 2
else:
# Fallback code block
```

Key Conditional Rules

1. Only one matching block within an `if-elif-else` ladder executes (see [3:18:20], [3:22:20]).
2. Multiple independent `if` statements evaluate separately from one another (see [3:26:30]).
3. `else` and `elif` statements cannot exist without an initial `if` statement (see [3:30:03]).

4. The `in` keyword checks if a element/substring exists inside a collection/string (e.g., `"spam" in message`) (see [3:43:06], [3:45:06]).

Important Concepts

• **Python Interpreter**: A program that converts human-readable Python script (.py) into executable machine code for hardware execution.
• **Pip**: The standard package manager used to install and manage external software packages/modules in Python.
• **REPL**: Stands for Read-Evaluate-Print Loop; an interactive command-line shell used to execute single Python lines instantly.
• **Variable Identifier**: The name assigned to identify a variable, function, or class within a program.
• **Typecasting**: The process of converting a variable's value from one data type to another (e.g., converting a string input into an integer).
• **Immutability**: A property of a data object (such as strings or tuples) preventing its contents from being changed or modified after assignment.
• **Mutability**: A property of a data structure (such as lists or dictionaries) allowing modification of its elements in place.
• **F-String**: A Python string formatting method prefixed with `f` allowing variables to be evaluated directly within `{}` inside the string.
• **Indentation**: Whitespace added at the beginning of code lines to designate code blocks under conditionals, functions, or loops.
• **Modulo Operator (`%`)**: An arithmetic operator that computes and returns the numerical remainder of a division operation between two numbers.

Video Chapters

• 00:00 Course Overview and Python Environment Setup
• 10:25 Chapter 1: Modules, Comments, and Pip
• 35:25 Chapter 2: Variables and Data Types
• 1:16:10 Chapter 3: Strings and String Functions
• 1:51:08 Chapter 4: Lists and Tuples
• 2:26:09 Chapter 5: Dictionaries and Sets
• 3:10:02 Chapter 6: Conditional Expressions
• 3:59:58 Chapter 7: Introduction to Loops

Flashcards

Front · Back · Topic
What does REPL stand for in Python, and how does it work? · REPL stands for Read-Evaluate-Print-Loop. It reads the input code, evaluates it, prints the output in the terminal, and loops back to repeat the process. · Python Basics

What is a Python module and how do you install an external module? · A module is a file containing code written by someone else that you can import into your project. External modules are installed using Python's package manager, pip (e.g., `pip install pyjokes`). · Modules & Pip

What data type is returned by the `input()` function in Python? · The `input()` function always returns input as a string data type. If numeric input is required, it must be explicitly converted using typecasting functions like `int()` or `float()`. · Variables & Data Types

Why are Python strings considered 'immutable'? · Strings are immutable because individual characters inside an existing string cannot be changed. Running string methods like `.replace()` generates and returns a new string rather than modifying the original one. · Strings

What is the key difference between a list and a tuple in Python? · Lists are mutable and can be altered in place using methods like `sort()` or `append()`, whereas tuples are immutable and their elements cannot be changed once defined. · Data Structures

How do you create an empty set in Python? · An empty set must be defined using `s = set()`. Writing `s = {}` creates an empty dictionary instead of a set. · Sets & Dictionaries

What is the difference between `dict.get('key')` and `dict['key']`? · If the specified key is not present in the dictionary, `dict.get('key')` returns `None`, whereas `dict['key']` throws a KeyError. · Sets & Dictionaries

How does Python define code blocks inside `if` statements? · Python uses indentation (leading spaces) to specify code blocks inside statements like `if`, `elif`, or `else`. Lines indented at the same level belong to that specific control block. · Conditionals